

Transport Layer: Multiplexing and Demultiplexing

1 Concept Overview

The **Transport Layer** provides a **process-to-process communication service**, extending the host-to-host delivery of the network layer. This is achieved through two key functions:

- **Multiplexing:** Collects data from multiple processes, adds transport-layer headers (including port numbers), and forwards the segments to the network layer.
- **Demultiplexing:** Receives incoming transport-layer segments and delivers them to the correct process based on port numbers.

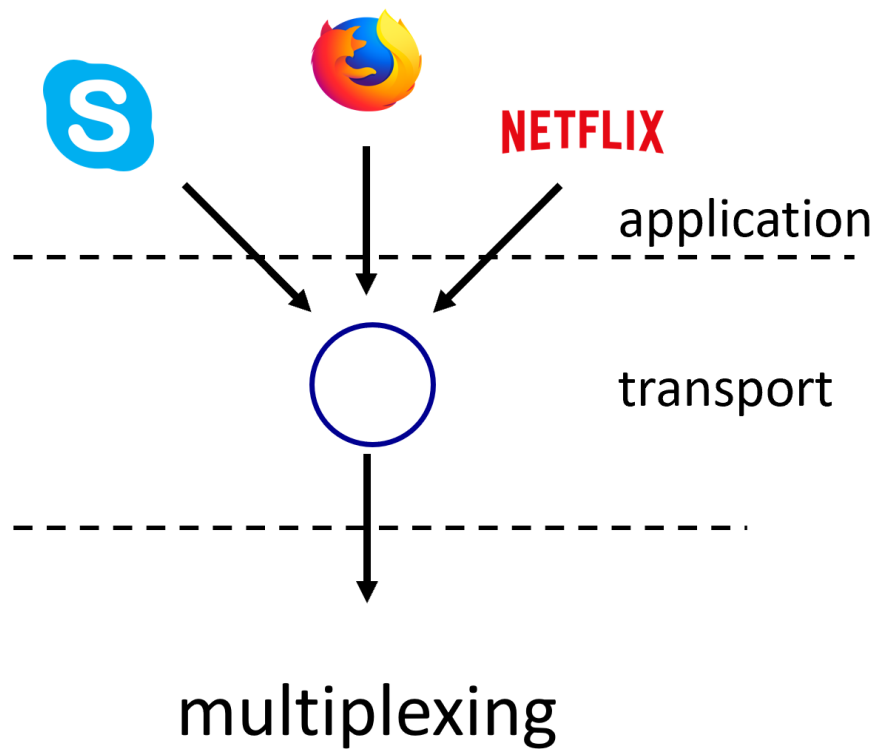
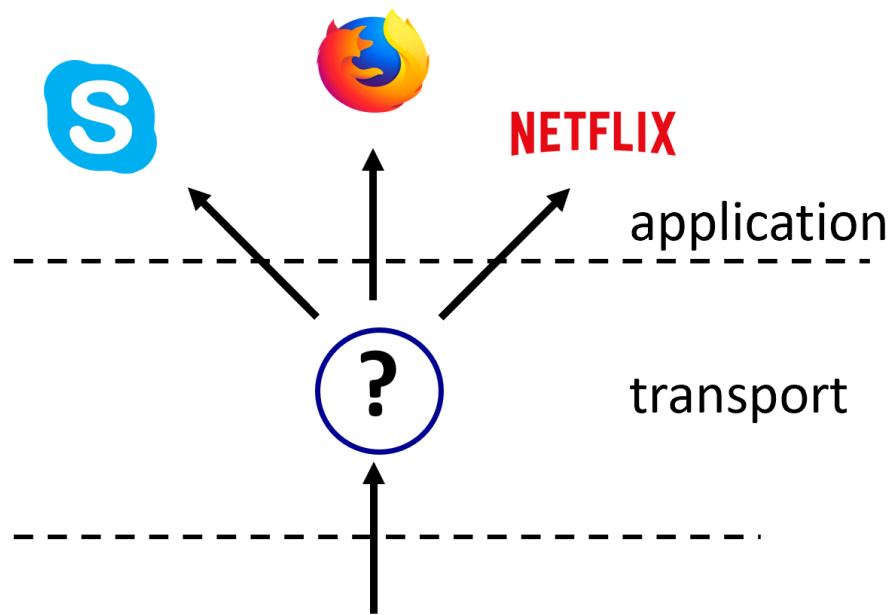


Figure 1: Multiplexing at the sender



de-multiplexing

Figure 2: Demultiplexing at the receiver

Note

Multiplexing and demultiplexing are fundamental because multiple applications often share a single host, requiring the transport layer to correctly identify which process should receive which data.

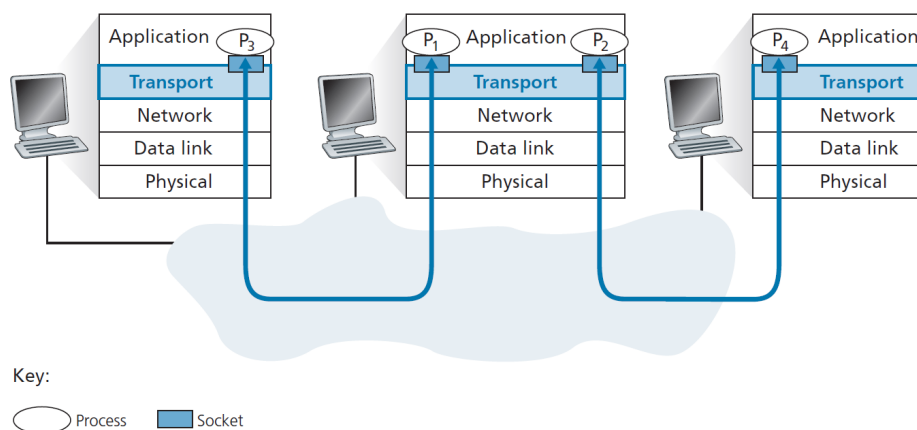


Figure 3: Combined illustration of multiplexing and demultiplexing

2 Analogy: Postal Service and Families

Example

- A **mail carrier** delivers all letters to a household $\Rightarrow$ analogous to the network layer delivering data to the host.
- **Bill** inspects each letter's recipient and hands it to the correct sibling $\Rightarrow$ demultiplexing at the transport layer.
- **Ann** collects outgoing letters from siblings and hands them to the mail carrier $\Rightarrow$ multiplexing at the transport layer.

3 Sockets and Port Numbers

- Applications use **sockets** as communication endpoints.
- Each socket is uniquely identified using a port number.
- **UDP socket:** Identified by (Destination IP, Destination Port).
- **TCP socket:** Identified by a **four-tuple**:

(Source IP, Source Port, Destination IP, Destination Port)

Port Number Ranges

Range	Usage
0 – 1023	Well-known ports (HTTP 80, FTP 21, DNS 53, SMTP 25)
1024 – 49151	Registered ports (assigned to specific services)
49152 – 65535	Dynamic/private ports (ephemeral, assigned by OS)

Important

Servers use fixed **well-known ports** for standard services, while clients usually use ephemeral ports automatically assigned by the OS.

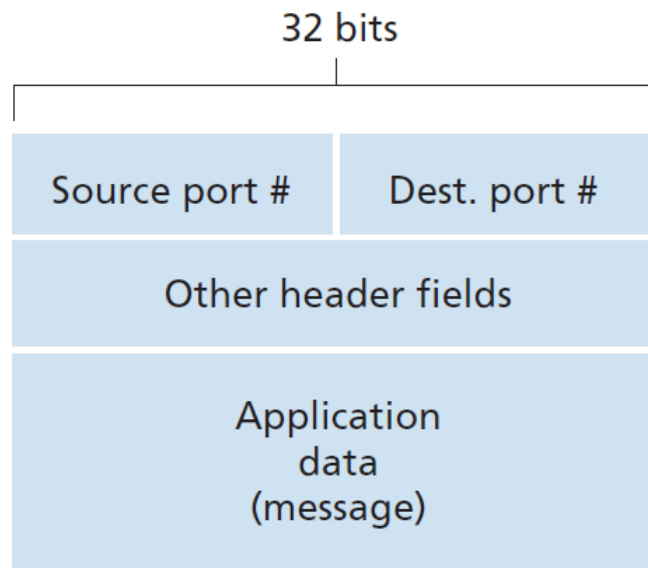


Figure 4: Source and destination port fields in a transport-layer segment

4 UDP Multiplexing and Demultiplexing

- UDP is **connectionless**: each socket is identified only by its destination port.
- Multiple sources can send data to the same UDP port.
- The **source port** acts as a return address for replies.

Example

Host A (port 19157) sends data to Host B (port 46428). Host B's transport layer examines the destination port (46428) and directs the segment to the correct socket. Replies from B use the incoming segment's source port (19157) as the destination port.

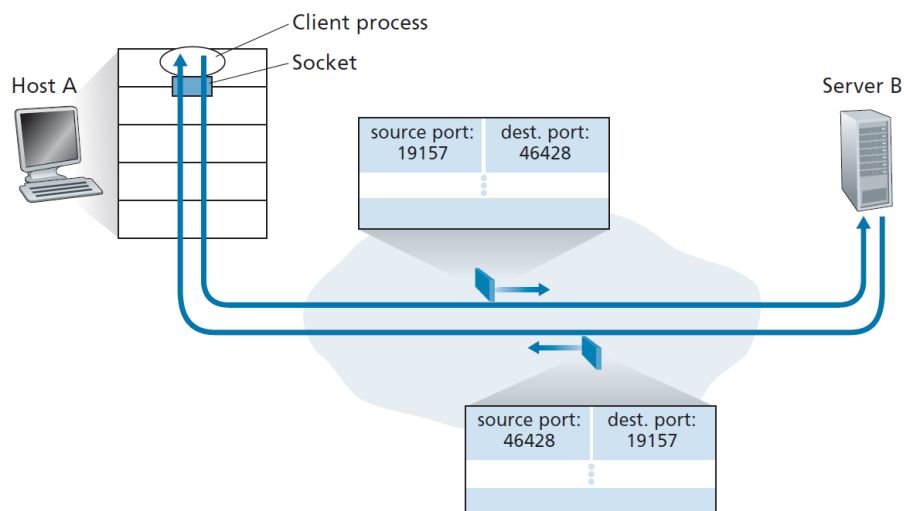


Figure 5: Source and destination ports invert in replies

Connectionless Demultiplexing Example

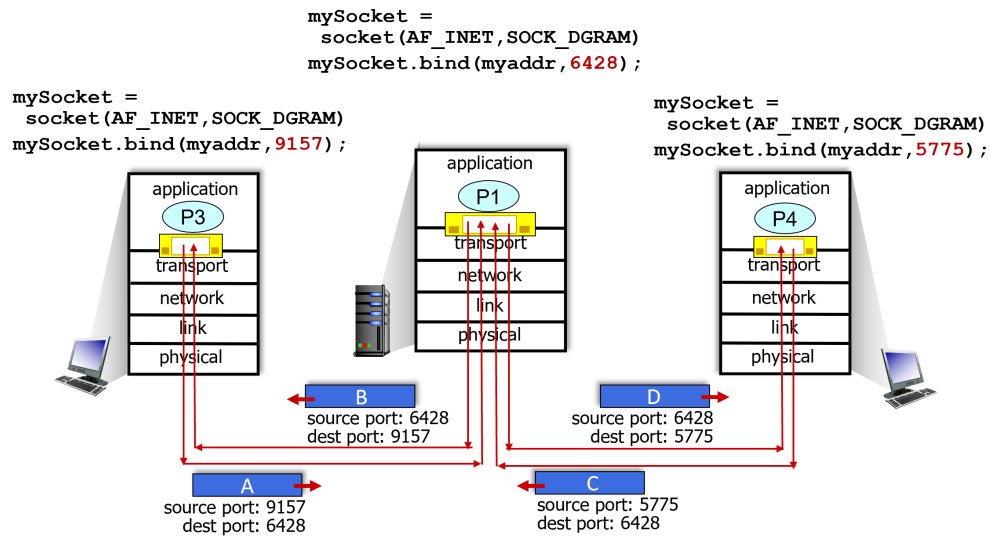


Figure 6: UDP connectionless demultiplexing

Socket Bindings

```

mySocket = socket(AF_INET, SOCK_DGRAM)
mySocket.bind(myaddr, 9157)

```

```

mySocket = socket(AF_INET, SOCK_DGRAM)
mySocket.bind(myaddr, 6428)

```

```

mySocket = socket(AF_INET, SOCK_DGRAM)
mySocket.bind(myaddr, 5775)

```

- $P_1 \rightarrow$ port 9157
- $P_2 \rightarrow$ port 6428
- $P_3 \rightarrow$ port 5775

Demultiplexing Behavior

- Incoming datagrams are delivered based only on **destination port**.
- Example mappings:
 - $DP = 9157 \rightarrow P_1$
 - $DP = 6428 \rightarrow P_2$

– $DP = 5775 \rightarrow P_3$

- Replies are sent using the source information:
 - From P_1 to $(IP_2, 6428)$ if sender was P_2 .
 - From P_1 to $(IP_3, 5775)$ if sender was P_3 .

Example

Even though P_1 listens on a single port (9157), it can handle datagrams from multiple clients. UDP treats each datagram independently.

5 TCP Multiplexing and Demultiplexing

- TCP is **connection-oriented**: each socket is identified by a **four-tuple**.
- This enables multiple simultaneous client connections to the same server port.
- Example: Web servers like Apache always use port 80, but distinguish clients by IP and source port.

Note

TCP provides reliable communication and uses demultiplexing to separate simultaneous client sessions.

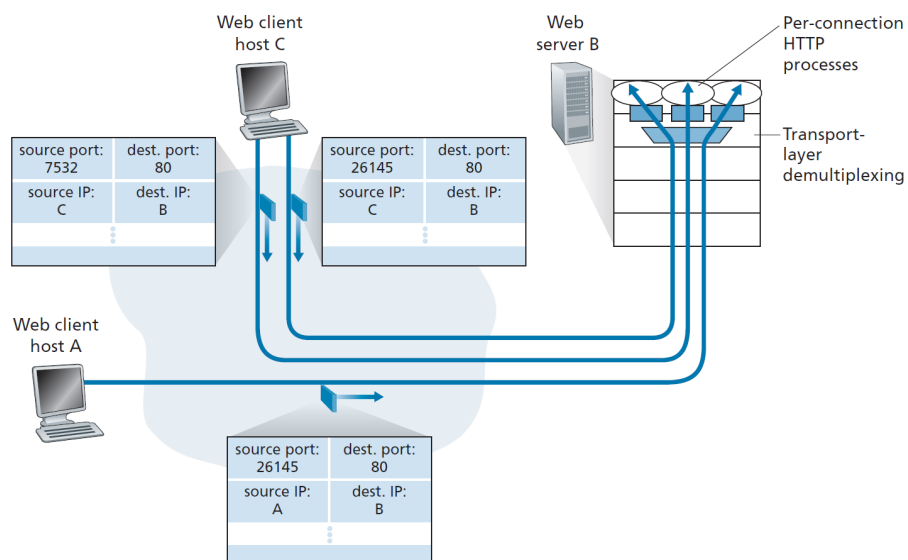
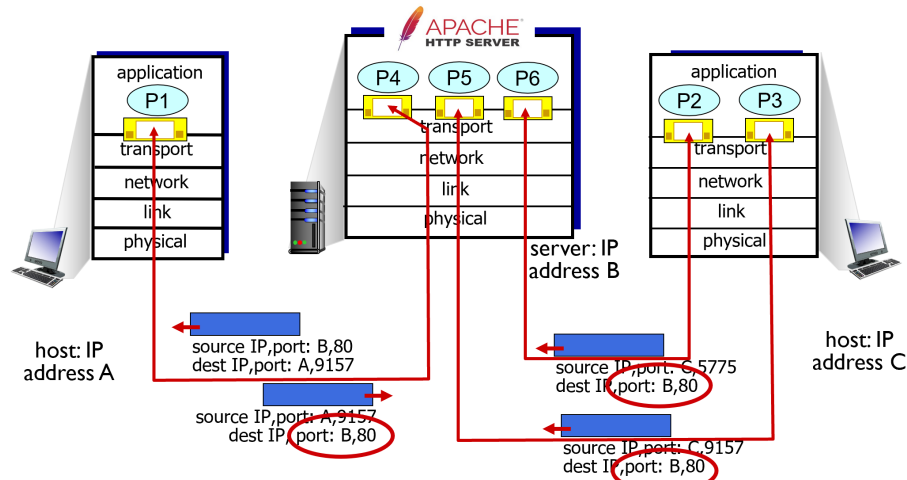


Figure 7: Two clients using the same server port (80) simultaneously

Connection-Oriented Demultiplexing Example



Three segments, all destined to IP address: B,
dest port: 80 are demultiplexed to *different* sockets

Figure 8: TCP connection-oriented demultiplexing (Apache Server)

Principle: A TCP socket is identified by:

$$(\text{SrcIP}, \text{SrcPort}, \text{DstIP}, \text{DstPort})$$

Example Setup

- Server *B*, port 80 (Apache server).
- Client *A* and Client *C* initiate connections.

Communication Flows

Segment	Source IP	Source Port	Dest IP	Dest Port
S_1	<i>A</i>	9157	<i>B</i>	80
S_2	<i>C</i>	5775	<i>B</i>	80
S_3	<i>C</i>	9157	<i>B</i>	80

Server Demultiplexing

Segment	Socket Quadruple	Process
S_1	$(A, 9157, B, 80)$	P_4
S_2	$(C, 5775, B, 80)$	P_5
S_3	$(C, 9157, B, 80)$	P_6

Conclusion

Even though all three segments target $(B, 80)$, their distinct source tuples create unique TCP connections:

$$S_1 \rightarrow P_4, \quad S_2 \rightarrow P_5, \quad S_3 \rightarrow P_6$$

6 Port Scanning

- Administrators use tools like **nmap** to detect open ports and services.
- Attackers can exploit open/vulnerable ports.

Important

Open ports must be secured. Example: The Slammer worm exploited SQL Server via UDP port 1434.

7 Key Takeaways

- **Multiplexing:** Gather outgoing data from multiple sockets.
- **Demultiplexing:** Deliver incoming data to correct sockets.
- **UDP sockets:** Identified by (Dest IP, Dest Port).
- **TCP sockets:** Identified by (Src IP, Src Port, Dst IP, Dst Port).
- **Well-known ports:** Reserved for standard services.
- **Ephemeral ports:** Temporary, OS-assigned.
- Open ports $\Rightarrow$ both administrative use and security risk.